



**PERIYAR
MANIAMMAI**
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University)
Established Under Sec. 3 of UGC Act, 1956 | NAAC Accredited
think • innovate • transform

DEPARTMENT OF COMPUTER SCIENCE

ASSIGNMENT - I

NAME : **A. MOHAMED SULAIMAN**

REGISTER NUMBER : **124012173089**

CLASS : **B.Sc., ARTIFICIAL INTELLIGENCE**

SEMESTER : **V**

COURSE NAME : **NEURAL NETWORKS**

COURSE CODE : **U24AI303**

TOPIC : **FUNDAMENTAL OF NEURAL NETWORK AND
LAYERS OF NEURAL NETWORK**

DATE OF SUBMISSION : **31/08/2026**

ACTIVATION FUNCTIONS AND TERMINOLOGY OF NEURAL NETWORKS

1. Introduction

A Neural Network is a machine learning model inspired by the structure and working of the human brain. It consists of interconnected processing units called **neurons**. Neural networks are widely used in image recognition, speech recognition, prediction, classification, and natural language processing.

An important component of a neural network is the **activation function**. It determines whether and how strongly a neuron should be activated.

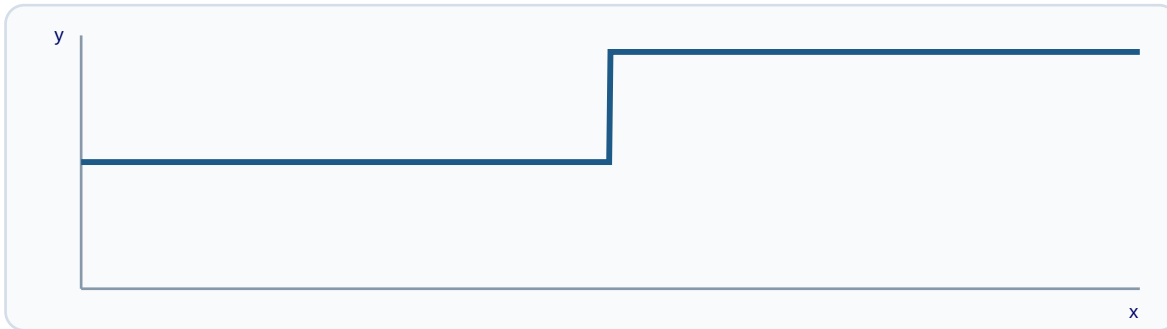
Key idea: activation functions introduce non-linearity, allowing a neural network to learn complex relationships rather than only simple linear mappings.

2. Types of Activation Functions

2.1 Binary Step Function

$$f(x) = 1, \text{ if } x \geq 0$$
$$f(x) = 0, \text{ if } x < 0$$

- Simple activation function.
- Produces binary output.
- Mainly used in simple perceptron models.
- Not suitable for modern deep neural networks because it is not differentiable.

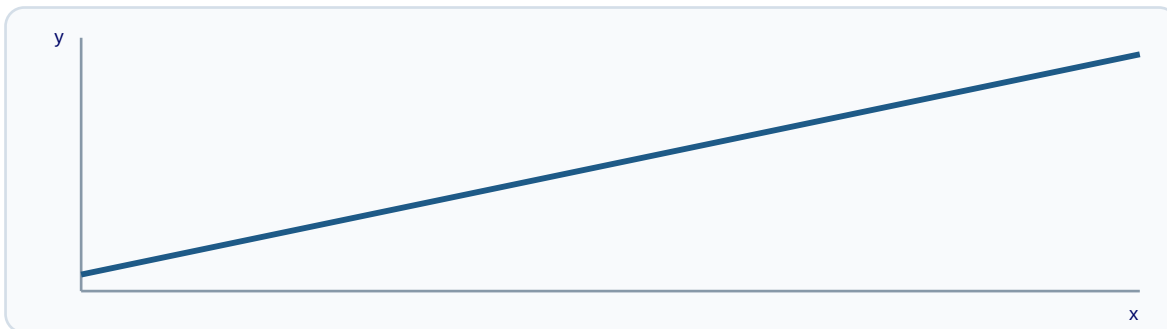


Graph: 2.1 Binary Step Function

2.2 Linear Activation Function

$$f(x) = x$$

- Simple and easy to understand.
- Output can have any real value.
- Mostly used in the output layer for regression problems.
- Not suitable for hidden layers because multiple linear layers still behave like one linear transformation.



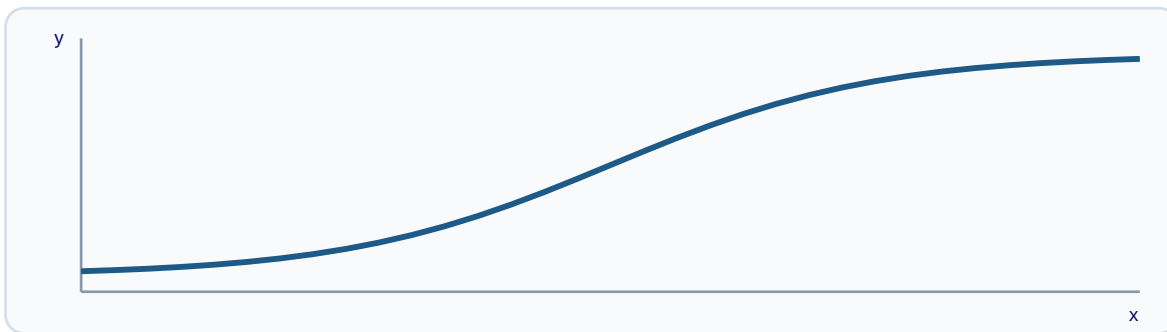
Graph: 2.2 Linear Activation Function

Activation Functions (continued)

2.3 Sigmoid Function

$$f(x) = 1 / (1 + e^{-x})$$

- Smooth and differentiable.
- Useful for binary classification output.
- Output is between 0 and 1.
- Can suffer from the vanishing gradient problem.

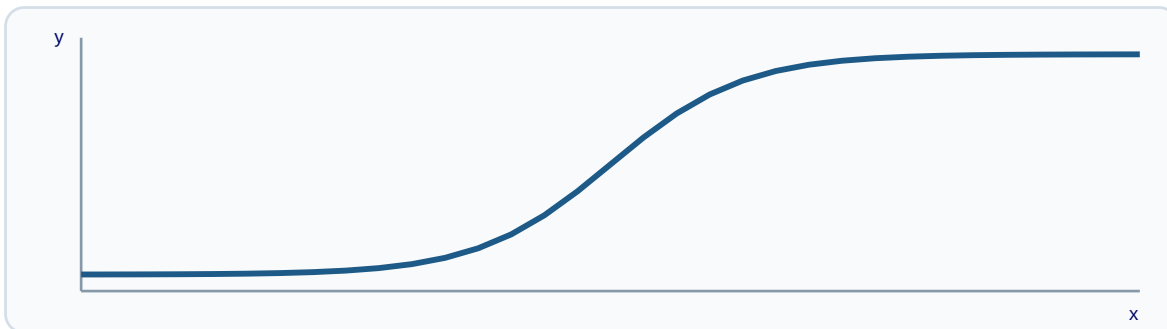


Graph: 2.3 Sigmoid Function

2.4 Tanh Function

$$f(x) = \tanh(x)$$

- Zero-centered.
- Useful in some neural network architectures.
- Usually performs better than sigmoid in hidden layers.
- Can also suffer from the vanishing gradient problem.



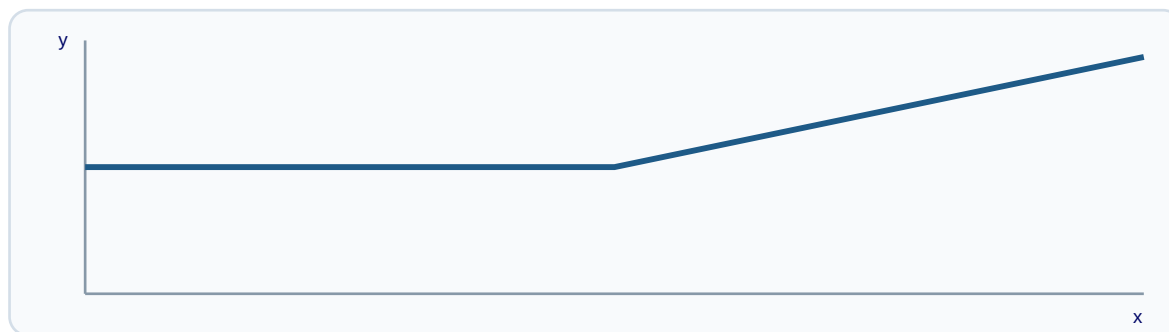
Graph: 2.4 Tanh Function

Activation Functions (continued)

2.5 ReLU Function

$$f(x) = \max(0, x)$$

- One of the most commonly used activation functions in deep learning.
- Output is 0 for negative inputs.
- Output equals the input for positive values.
- Computationally simple.
- Helps reduce the vanishing gradient problem.
- Can suffer from the dying ReLU problem.



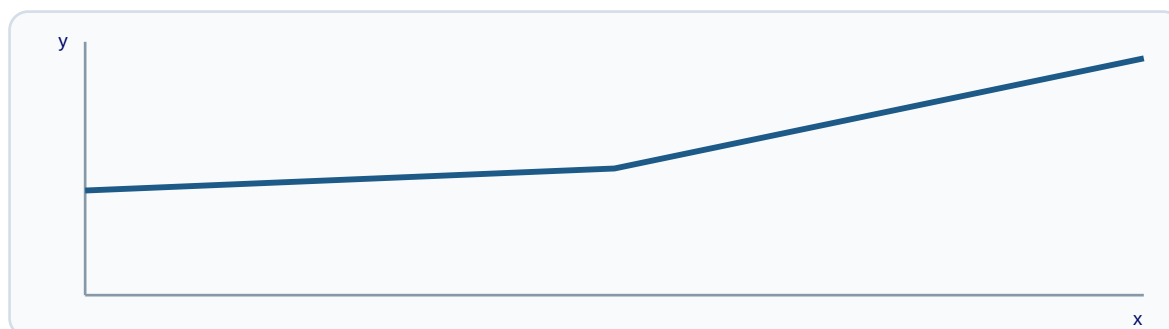
Graph: 2.5 ReLU Function

2.6 Leaky ReLU

$$f(x) = x, \text{ if } x > 0$$
$$f(x) = \alpha x, \text{ if } x \leq 0$$

where α is a small positive value.

- Reduces the dying ReLU problem.
- Allows a small gradient for negative inputs.
- Useful in deep neural networks.



Graph: 2.6 Leaky ReLU

2.7 Softmax Function

The Softmax function is mainly used in the output layer for **multi-class classification**. It converts a group of values into probabilities whose total is 1.

$$f(x_i) = e^{x_i} / \sum e^{x_j}$$

- Produces probability values between 0 and 1.
- Sum of all output probabilities is 1.
- Commonly used for classification into multiple classes.

Example prediction

Class	Probability
Cat	0.70
Dog	0.20
Bird	0.10
Total	1.00

In the example above, the model assigns the highest probability to **Cat**, while the three class probabilities add up to 1.00.

3. Activation Function Graphs & Comparison

The major activation functions can be represented using characteristic graph shapes. The following table summarizes their output ranges and common uses.

Activation Function	Output Range	Common Use
Binary Step	0 or 1	Simple perceptron
Linear	$-\infty$ to $+\infty$	Regression output
Sigmoid	0 to 1	Binary classification
Tanh	-1 to +1	Hidden layers / some sequence models
ReLU	0 to $+\infty$	Deep neural networks
Leaky ReLU	$-\infty$ to $+\infty$	Deep neural networks
Softmax	0 to 1	Multi-class classification

A practical rule of thumb:

- **ReLU / Leaky ReLU:** common choices for hidden layers in deep networks.
- **Sigmoid:** useful when the final output represents a binary probability.
- **Softmax:** useful when the final output represents probabilities over multiple classes.
- **Linear:** commonly used for regression outputs.

4. Terminology of Neural Networks

Neural Network Terminology — Part I

4.1 Neuron

A neuron is the basic processing unit of a neural network. It receives inputs, applies weights and bias, and passes the result through an activation function.

4.2 Input

Input refers to the information given to a neural network. Example: in an image classification system, the pixel values of an image can be used as inputs.

4.3 Weight

A weight determines the importance of an input to a neuron. During training, the network adjusts weights to improve its predictions.

4.4 Bias

A bias is an additional parameter added to the weighted sum of inputs. It helps the neuron shift its activation function.

4.5 Activation Function

An activation function determines the output of a neuron based on its input. It also introduces non-linearity, allowing neural networks to learn complex patterns.

4.6 Input Layer

The input layer is the first layer of a neural network. It receives the input features from the dataset.

4.7 Hidden Layer

A hidden layer exists between the input and output layers. It performs computations and learns patterns from the input data. A neural network can have one or many hidden layers.

4.8 Output Layer

The output layer is the final layer. It produces the prediction or result. Examples: Regression → numerical value; Binary classification → probability/class; Multi-class classification → class probabilities.

4. Terminology of Neural Networks

Neural Network Terminology — Part II

4.9 Forward Propagation

Forward propagation is the process of passing input data through the neural network from the input layer to the output layer to produce a prediction.

4.10 Loss Function

A loss function measures the difference between the predicted output and the actual output. A lower loss generally indicates that the model's predictions are closer to the target values.

4.11 Backpropagation

Backpropagation is a training algorithm used to calculate how much each weight contributed to the error. The calculated gradients are then used to update the model's parameters.

4.12 Gradient

A gradient indicates the direction and rate at which the loss changes with respect to model parameters. Gradients are used to update weights during training.

4.13 Learning Rate

The learning rate controls how large the parameter updates are during training.

4.14 Epoch

An epoch means one complete pass through the entire training dataset. For example, if a model trains for 10 epochs, it processes the complete training dataset 10 times.

4.15 Batch Size

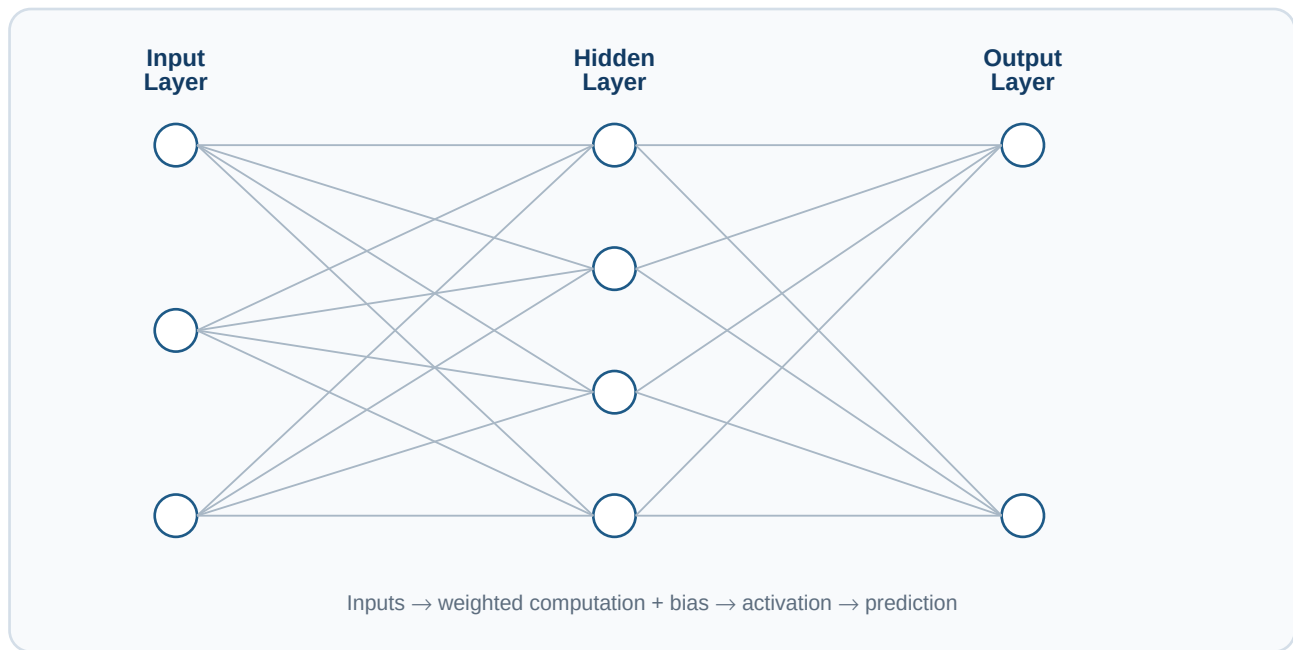
Batch size is the number of training examples processed before the model updates its parameters.

4.16 Overfitting

Overfitting occurs when a neural network learns the training data too closely and performs poorly on new, unseen data.

Neural Network Flow: From Input to Prediction

The terminology becomes easier to understand when viewed as a complete processing pipeline.



Basic neuron equation

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

The neuron first computes the weighted sum plus bias. The activation function then transforms this value before the signal is passed to the next layer.

Neural Network Training Process

A typical training cycle can be summarized as follows:

1	Input data	Training examples are supplied to the input layer.
2	Forward propagation	The data moves through the network to produce a prediction.
3	Loss calculation	The prediction is compared with the actual target using a loss function.
4	Backpropagation	Gradients are calculated to determine how model parameters contributed to the error.
5	Parameter update	Weights and bias are adjusted using an optimization method and the learning rate.
6	Repeat	The process continues over batches and epochs until the model learns useful patterns.

Learning-rate effect

- **Very high learning rate:** training may become unstable or overshoot useful solutions.
- **Very low learning rate:** training may be very slow.
- **Suitable learning rate:** balances stable learning with reasonable training speed.

Model Behaviour: Overfitting vs Underfitting

Understanding generalization is important because a model should learn patterns that work on unseen data—not simply memorize the training examples.

Concept	What happens?	Typical sign
Underfitting	Model is too simple or insufficiently trained to learn important patterns.	Poor performance on both training and unseen data.
Good fit	Model captures useful patterns and generalizes well.	Good training and testing performance.
Overfitting	Model learns the training data too closely, including noise.	Very good training performance but weaker testing performance.

Common hyperparameters

- Learning rate
- Number of hidden layers
- Number of neurons
- Batch size
- Number of epochs

5. Conclusion

Activation functions are an essential part of neural networks because they introduce non-linearity and help networks learn complex relationships. Different activation functions are suitable for different tasks.

Understanding neural-network terminology such as **neurons, weights, bias, layers, forward propagation, backpropagation, loss, gradients, and learning rate** is important for understanding how neural networks are designed and trained.

Quick Revision

Term	One-line meaning
Neuron	Basic processing unit of a neural network.
Weight	Controls the importance of an input.
Bias	Shifts the neuron's activation.
Activation	Adds non-linearity and transforms neuron output.
Loss	Measures prediction error.
Gradient	Shows how loss changes with parameters.
Epoch	One complete pass through the training dataset.
Batch size	Number of examples processed before an update.
Hyperparameter	Training setting chosen rather than directly learned.

— End of Assignment —